Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Data Structures and Libraries

Prof. Reza Skandari

Today

1. Recap
2. Python:
    - Dictionaries and sets
    - Modules and libraries
3. Accessing data on the web:
    - Scraping
    - APIs

Test your instincts

Poll time! Open menti.com and vote on each question.

What is the output?

```
x = 0
y = 3
z = y
while y >= 1:
    y -= 1
    x = x + y
print(x, y, z)
```

A. 6 0 3

B. 6 3 3

C. 5 1 1

D. 3 1 3

E. Something else

What is the output?

```
s = '01234'
t = 'y'
for c in s:
    t = t + t
print(len(t))
```

A. 10

B. 15

C. 16

D. 32

E. I don't know

What is the time complexity?

```
def mystery(a):
    for i in range(len(a)):
        smallest = sorted(a[i:])[0]
        j = a.index(smallest, i)
        a[i], a[j] = a[j], a[i]

    return a
```

A. O(n)

B. O(n log n)

C. O(n^2)

D. O(n^2 log n)

E. O(n^3)

What is the time complexity?

```
def selection_sort(a):
    for i in range(len(a)):
        smallest = min(a[i:])
        j = a.index(smallest, i)
        a[i], a[j] = a[j], a[i]

    return a
```

A. O(n)

B. O(n log n)

C. $O(n^2)$

D. $O(n^2 \log n)$

E. $O(n^3)$

What is the time complexity?

```
def mystery(matrix):
    for item in matrix:
        item.sort()

    return matrix

# matrix has n rows, each of length m
```

A. $O(n \cdot m)$

B. $O(m \log m)$

C. $O(n \cdot m \log m)$

D. $O(n^2 \log n)$

E. $O(n \log n)$

Today

1. Recap
2. **Python:**
    - **Dictionaries and sets**
    - Modules and libraries
3. Accessing data on the web:
    - Scraping
    - APIs

Lists are not always ideal

```
music_data = [
    ['Jose', 'Jimi Hendrix'],
    ['Iva', 'Midori Takada'],
    ['Misha', 'BTS'],
    ['Pat', 'Igor Stravinsky']
]
```

Or, keeping the two lists separate:

```
names = ['Jose', 'Iva', 'Misha', 'Pat']
favorite_music = ['Jimi Hendrix', 'Midori Takada', 'BTS', 'Igor Stravinsky']
```

How do we look up Misha's favorite music?

A dictionary maps keys to values

A dictionary allows easy lookups:

```
music_data = {
    'Jose': 'Jimi Hendrix',
    'Iva': 'Midori Takada',
    'Misha': 'BTS',
    'Pat': 'Igor Stravinsky'
}
music_data['Misha']  # access value directly with key
```

A list maps integer indices to values. A dictionary maps (immutable) **keys** (integer, string, ...) to values.

Working with dictionaries

```
music_data = {
    'Jose': 'Jimi Hendrix', 'Iva': 'Midori Takada',
    'Misha': 'BTS', 'Pat': 'Igor Stravinsky'
}
music_data['Valerie'] = 'Johnny Cash'
music_data['Iva'] = 'Steve Reich'

for key in music_data:
    print(music_data[key])

for key, value in music_data.items():
    print(key, value)

# If key 'Woonam' is not in the dictionary,
# return default value Bach
music_data.get('Woonam', 'J.S. Bach')
```

Sets are collections of unique, unordered items

```
berries = {'Raspberry', 'Cloudberry', 'Lingonberry'}
for berry in berries:
    print(berry)

berries.add('Blueberry')
berries.remove('Lingonberry')
print('Blueberry' in berries)

fruit = set()
fruit.add('Apple')
```

Big picture: how does a dictionary work?

Behind the scenes, a dictionary is really just a list (called a "hash table")

- Each key is turned into a number by a function, let's call it `index(key)`
- That number tells Python exactly which slot of the underlying list to use
- Each slot stores **both the key and the value**, not just the value (so Python can confirm it found the right key, and reconstruct `.items()`)

Looking a key up

- Looking up `music_data['Misha']` really means: "compute `index('Misha')`, then jump straight to that slot"
- No searching required: that's why dictionary lookups are $O(1)$ on average, versus $O(n)$ to search a list

Example (illustrative numbers, not real output):

```
index('Jose')  = 0   ->  table[0] = ('Jose', 'Jimi Hendrix')
index('Iva')   = 3   ->  table[3] = ('Iva', 'Midori Takada')
index('Misha') = 2   ->  table[2] = ('Misha', 'BTS')
index('Pat')   = 5   ->  table[5] = ('Pat', 'Igor Stravinsky')
```

Big picture: hashing in action

- This `index` function is called a **hash function**; Python exposes it directly as `hash(key)`
- Two different keys can land on the same slot (a **collision**): Python handles this automatically behind the scenes, so dictionaries just work
- This is why dictionary keys must be **immutable**:
  - If a key changed after being stored, `index(key)` would now point somewhere else
  - The entry would effectively be lost, stuck in the slot for its old value
- A **set** uses the same hashing idea, just storing keys with no associated values

Test your instincts

Poll time! Open menti.com and vote on each question.

After the code runs, what is in `desserts`?

```
desserts = {'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2}
desserts['Fruit'] = 5
desserts['Apple pie'] = 1
```

A. `{'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2, 'Apple pie': 1}`

B. `{'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 5, 'Apple pie': 1}`

C. `{'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2, 'Fruit': 5, 'Apple pie': 1}`

D. There will be an error

E. I don't know

After the code runs, what is in `desserts`?

```
desserts = {'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2}
new_orders = ['Tiramisu', 'Fruit']
for order in new_orders:
    desserts[order] += 1
```

A. `{'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2}`

B. `{'Cheesecake': 10, 'Tiramisu': 4, 'Fruit': 4}`

C. `{'Cheesecake': 10, 'Tiramisu': 4, 'Fruit': 3}`

D. There will be an error

E. I don't know

After the code runs, what is in `desserts`?

```
desserts = {'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2}
new_orders = ['Tiramisu', 'Fruit', 'Fruit', 'Tiramisu', 'Apple pie']
for order in new_orders:
    desserts[order] += 1
```

A. `{'Cheesecake': 10, 'Tiramisu': 3, 'Fruit': 2, 'Apple pie': 1}`

B. `{'Cheesecake': 10, 'Tiramisu': 5, 'Fruit': 4}`

C. `{'Cheesecake': 10, 'Tiramisu': 5, 'Fruit': 4, 'Apple pie': 1}`

D. There will be an error

E. I don't know

After the code runs, what is in `desserts`?

```
orders = ['Tiramisu', 'Fruit', 'Fruit', 'Tiramisu', 'Apple pie']
desserts = set(orders)
```

A. `{'Cheesecake', 'Tiramisu', 'Fruit', 'Apple pie'}`

B. `{'Tiramisu', 'Fruit', 'Fruit', 'Tiramisu', 'Apple pie'}`

C. `{'Tiramisu', 'Fruit', 'Apple pie'}`

D. There will be an error

E. I don't know

Today

1. Recap
2. **Python:**
    - Dictionaries and sets
    - **Modules and libraries**
3. Accessing data on the web:
    - Scraping
    - APIs

Dividing code into modules

`my_algorithms.py`

```
def lin_search(L, x):
    for elem in L:
        if elem == x:
            return True
    return False
print('Running my_algorithms.py')
```

`working.py`

```
import my_algorithms as alg  # "as" part is optional
from my_algorithms import lin_search  # specific function

L = [1, 2, 2, -2, 9]
found_10 = alg.lin_search(L, 10)
found_2 = lin_search(L, 2)
```

When you import, Python will run all of `my_algorithms.py`

Using code from libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

There are thousands of Python libraries:

- Some common packages are preinstalled.
- NumPy, pandas, matplotlib.
- Check what's installed: `pip list`
- Install a package: `pip install package_name`
- Upgrade a package: `pip install --upgrade package_name`

Today

1. Recap
2. Python:
    - Dictionaries and sets
    - Modules and libraries
3. **Accessing data on the web:**
    - **Scraping**
    - **APIs**

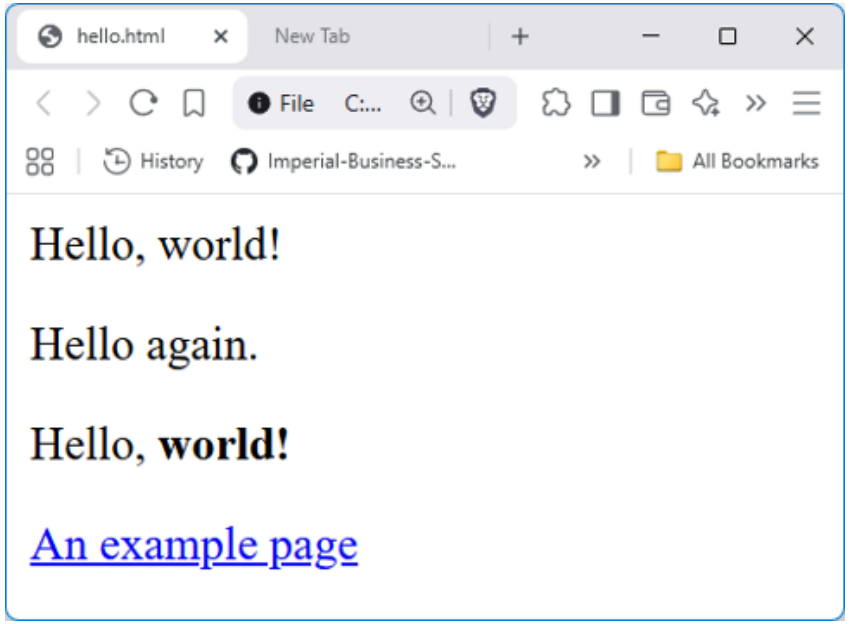
Accessing data on the web

- The web is full of data and services
- Quick and repeated access could be useful for:
  - Analyzing tweets/government spending
  - Getting data to/from Google Maps/AWS/Bloomberg?
  - Finding an apartment in London/best coffee close to you?

We can use APIs or scraping for access

HTML

HTML: HyperText Markup Language




```
<p>Hello, world!</p>
<p>Hello again.</p>
<p>Hello, <strong>world!</strong></p>
<p><a href="http://www.example.com/">An example page</a></p>
```

HTML tags

Tags allow you to format content display

```
<p>Hello, world!</p>
<p>Hello again.</p>
<p>Hello, <strong>world!</strong></p>
<p><a href="http://www.example.com/">An example page</a></p>
```

- Tags are within angle brackets.
  - `<p>` is the starting tag and `</p>` is the ending tag.
  - Inside there is the paragraph content.
- Tags can be nested:
  - `<p>` and `<strong>` are parent and child tags.
  - Two `<p>` tags are sibling tags.

HTML tags

Tags can have attributes

```
<p>Hello, world!</p>
<p>Hello again.</p>
<p>Hello, <strong>world!</strong></p>
<p><a href="http://www.example.com/">An example page</a></p>
```

Example:

- Hyperlink tag `<a>` specifies the target URL as the `href` attribute in the opening tag.

CSS: Cascading Style Sheets

Web sites use custom attributes to make content look fancy with CSS

Styling targets (selectors):

- id
- class
- other selectors (e.g., tag names: buttons)

CSS Example

```css
/* Style by tag name */
p {
  font-size: 16px;
  color: #333333;
}

/* Style by class */
.highlight {
  background-color: yellow;
  font-weight: bold;
}

/* Style by id */
#header {
  font-size: 24px;
  text-align: center;
}
```

Parsing HTML with Python

- Websites use HTML tag structure to look nice
  - We can exploit these tags to parse their contents
  - Find all links in a web page through `<a href...>`
  - Great libraries exist!
- Beautiful Soup 4 (BS)
  - Detective work to identify what we want in the page (right tags)
  - Looping through tags with BS

Parsing HTML with Python

```
from bs4 import BeautifulSoup
html = '<p>Hello, world!</p> <p>Hello again.</p>'
soup = BeautifulSoup(html, 'html.parser')  # text and the parser BS should use
print(soup.text)
```

Finding things in soup

The `soup` variable contains a BeautifulSoup object; `soup.text` contains the text content.

```
>>> paragraphs = soup.find_all('p')  # find all <p> tags

>>> paragraphs
[<p>Hello, world!</p>, <p>Hello again.</p>]

>>> type(paragraphs)  # augmented version of a list
bs4.element.ResultSet

>>> first = paragraphs[0]
>>> first.name
'p'
>>> first.text
'Hello, world!'

>>> type(first)
bs4.element.Tag
```

Browsing with Python: the `requests` library

```
import requests
r = requests.get('http://www.example.com')

r.ok    # was the access attempt successful
r.text  # raw HTML
```

Requests + Soup

Pull data and process:

```
import requests
from bs4 import BeautifulSoup
r = requests.get('http://www.example.com')
soup = BeautifulSoup(r.text, 'html.parser')
links = soup.find_all('a')
```

Analyze the results:

```
>>> first_link = links[0]

>>> first_link.text
'More information...'
>>> first_link.attrs  # dictionary of attributes
{'href': 'http://www.iana.org/domains/example'}

>>> first_link.attrs['href']
'http://www.iana.org/domains/example'
```

Soup workflow

- Use `requests` to get HTML
- Create a soup object with BS
- Inspect the HTML in browser to find the tags you need
- BS: `find` (first) and `find_all` methods are often useful
- BS: `text` and `attrs` attributes are often useful
- Get data into a Python data structure (list, dictionary, ...)
- Save to a file

BS has great documentation online.

Scraping ethics

Websites don't always like scraping:

- Be polite!
- Check the terms for access
- Pace your requests

```
from time import sleep
sleep(1)
```

API: better to use APIs if available.

- Bonus: more robust access and cleaner data

API: Application Programming Interface

- A controlled way to access a service. Widely used:
  - Google Maps, TfL, Spotify, NY Times, Gmail, ...
- Easy and robust access to data in a nice form
- Typically need to register to use
- Limitations: types of data, access frequency

Data format:

- Often in JSON or XML format
- Both forms are common for semi-structured data
- Worth getting to know for analytics
- Great Python libraries for parsing both

API via Python

```
import requests
import pandas as pd

# API endpoint
url = ("https://api.datausa.io/tesseract/data.jsonrecords?"
       "cube=acs_yg_total_population_5&measures=Population"
       "&include=Year:2023&drilldowns=State,Year&limit=3,0")

# Make request
response = requests.get(url)

# Convert to JSON
data = response.json()

# Extract records
df = pd.DataFrame(data["data"])

# Show first rows
print(df.head())
```

JSON


```
{
  "annotations": {
    "subtopic": "Demographics",
    "source_name": "Census Bureau",
    "table_id": "B01003"
  },
  "page": { "limit": 3, "offset": 0, "total": 52 },
  "columns": ["State ID", "State", "Year", "Population"],
  "data": [
    {
      "State ID": "04000US01",
      "State": "Alabama",
      "Year": 2023,
      "Population": 5054253.0
    },
    {
      "State ID": "04000US02",
      "State": "Alaska",
      "Year": 2023,
      "Population": 733971.0
    },
    {
      "State ID": "04000US04",
      "State": "Arizona",
      "Year": 2023,
      "Population": 7268175.0
    }
  ]
}
```

Review

Today we covered:

- **Dictionaries and sets**: fast lookups by key, unique unordered collections
- **Modules and libraries**: organising code, using third-party packages
- **Accessing data on the web**: scraping with `requests` + BeautifulSoup, and APIs returning JSON

Up next:

- Session 6 workshop
- Session 7 prep — object-oriented programming